



Spring Boot

AUTOMATIZE SUAS TAREFAS

SPRINGBOOT SHEDULE:

<https://github.com/Cristianomeneses2008/springboot-schedule>

Palestra: Cristiano Meneses – 2026

Java Spring Boot

é um framework open source usado para facilitar as configurações de uma aplicação. Ele deriva do Spring, que também é um framework criado em 2003 por Rod Johnson, empresa por trás é a Pivotal.

Em abril de 2014, após 18 meses de desenvolvimento, testes e amadurecimento, a Pivotal entrega o Spring Boot 1.0. Hoje, a última versão estável em produção é a 3.2.0 e necessita no mínimo do Java 8 e já estamos Java 17 e 25.

O seu objetivo é claro: Facilitar o trabalho de configuração e proporcionar ao desenvolvedor que sua aplicação seja publicada o mais rápido possível.

O Java Spring Boot (Spring Boot) é uma ferramenta que facilita e agiliza o desenvolvimento de aplicativos da web e de microsserviços com o Spring Framework por meio de três principais recursos:

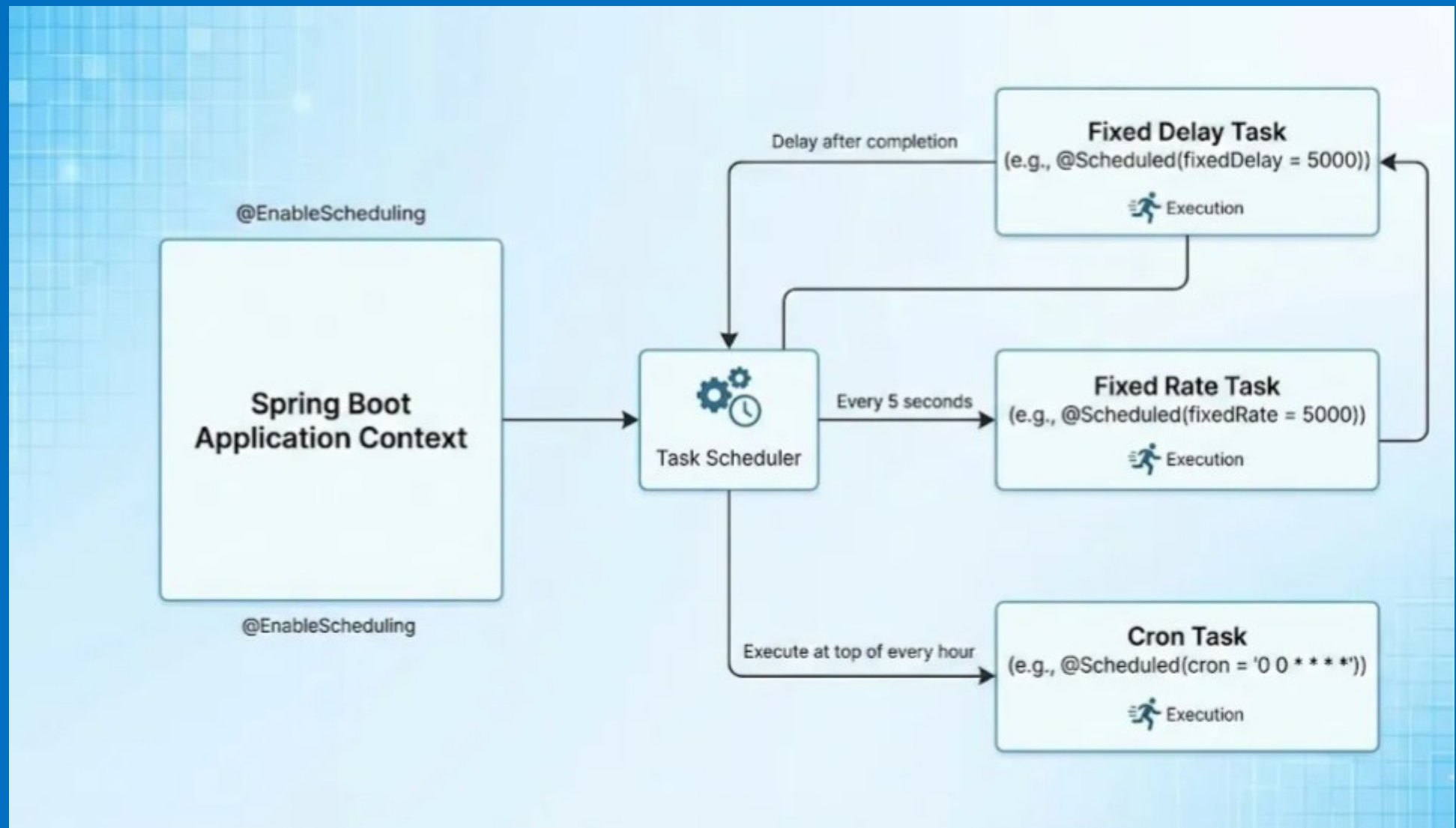
- 1. Autoconfiguração**
- 2. Uma abordagem opinativa à configuração**
- 3. A capacidade de criar aplicativos independentes**

Esses recursos trabalham juntos para oferecer uma ferramenta que permite configurar um aplicativo baseado em Spring com poucos requisitos de instalação e configuração.

A **Pivotal Software**, Inc. era uma empresa multinacional americana de software e serviços com sede em São Francisco que fornecia serviços de consultoria e hospedagem de plataforma em nuvem. Desde dezembro de 2019, a Pivotal faz parte da **VMware**.



Arquitetura do Schedule



Entendendo a sintaxe do Cron

As expressões cron no Spring usam seis campos (com um sétimo opcional para o ano).

```
segundo minuto hora dia mês dia da semana
```

Exemplos:

- `0 0 * * * *` - A cada hora, sempre no início de cada hora.
- `0 */15 * * * *` - A cada 15 minutos
- `0 0 12 * * *` - Todos os dias ao meio-dia
- `0 0 9 1 * *` - Às 9h da manhã do primeiro dia de cada mês
- `0 0 0 * * SUN` - Todos os domingos à meia-noite

Programação de Tarifas Fixas

Executar uma tarefa em intervalos fixos, independentemente da duração da execução anterior

```
@Scheduled(fixedRate = 60000)
```

```
public void reportCurrentTime () {
```

```
    System.out.println( "Hora atual: " + new Date ());
```

```
}
```



Agendamento com atraso fixo

Executar uma tarefa com um atraso fixo entre o fim de uma execução e o início da próxima

```
@Scheduled(fixedDelay = 60000)
public void processWithDelay () {
    System.out.println( "Processando com atraso fixo" );
    // Simula o trabalho
    Thread.sleep( 3000 );
}
```



FIM

Fontes de Pesquisa:

<https://spring.io/>

<https://www.ibm.com/br-pt/topics/java-spring-boot>